

λ Lambda

(parameter) -> { anweisung; }

Lambda – Motivation

Früher, ohne Lambda, war alles schlimmer! Wenn man „mal eben“ eine Collection umsortieren wollte, war das ein ziemlicher Akt! 🤯

```
public static void main(String[] args) {  
  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Alan", "Turing"),  
        new Person("Grace", "Hopper"),  
        new Person("Linus", "Torvalds"),  
        new Person("Adele", "Goldberg")));  
  
    persons.sort(new SortByFirstnameComparator());  
    System.out.println(persons);  
  
    persons.sort(new SortByLastnameComparator());  
    System.out.println(persons);  
}
```

🤯 Für jede neue Art der Sortierung musste eine neue Klasse erstellt und das Interface implementiert werden!

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

```
import java.util.Comparator;  
  
class SortByFirstnameComparator implements Comparator<Person> {  
    @Override  
    public int compare(Person p1, Person p2) {  
        return p1.firstname().compareTo(p2.firstname());  
    }  
}
```

Verwende die vorhandene compareTo-Methode in der Klasse String.

```
class SortByLastnameComparator implements Comparator<Person> {  
    @Override  
    public int compare(Person p1, Person p2) {  
        return p1.lastname().compareTo(p2.lastname());  
    }  
}
```

Ausgabe:

```
[Adele Goldberg, Alan Turing, Grace Hopper, Linus Torvalds]  
[Adele Goldberg, Grace Hopper, Linus Torvalds, Alan Turing]
```

Anonyme Klassen

Durch **anonyme Klassen** musste man nicht länger jedes Mal eine neue Klasse erstellen, um ein Interface zu implementieren. Aber schön ist das trotzdem noch nicht... 🙄

```
public static void main(String[] args) {  
  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Alan", "Turing"),  
        new Person("Grace", "Hopper"),  
        new Person("Linus", "Torvalds"),  
        new Person("Adele", "Goldberg")));  
  
    persons.sort(new Comparator<Person>() {  
        @Override  
        public int compare(Person p1, Person p2) {  
            return p1.firstname().compareTo(p2.firstname());  
        }  
    });  
    System.out.println(persons);  
  
    persons.sort(new Comparator<Person>() {  
        @Override  
        public int compare(Person p1, Person p2) {  
            return p1.lastname().compareTo(p2.lastname());  
        }  
    });  
    System.out.println(persons);  
}
```

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

Ausgabe:

```
[Adele Goldberg, Alan Turing, Grace Hopper, Linus Torvalds]  
[Adele Goldberg, Grace Hopper, Linus Torvalds, Alan Turing]
```

Lambda to the rescue!

Mit **Lambdas** kann man „mal eben“ eine Implementation für ein Interface erzeugen und als Parameter übergeben. 😊

```
public static void main(String[] args) {  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Alan", "Turing"),  
        new Person("Grace", "Hopper"),  
        new Person("Linus", "Torvalds"),  
        new Person("Adele", "Goldberg")));  
  
    persons.sort((p1, p2) -> p1.firstname().compareTo(p2.firstname()));  
    System.out.println(persons);  
  
    persons.sort((p1, p2) -> p1.lastname().compareTo(p2.lastname()));  
    System.out.println(persons);  
}
```

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

Ausgabe:

```
[Adele Goldberg, Alan Turing, Grace Hopper, Linus Torvalds]  
[Adele Goldberg, Grace Hopper, Linus Torvalds, Alan Turing]
```

Lambda – Definition

- Ein Lambda-Ausdruck ist ein (kurzer) Codeblock, der
 - optional Parameter entgegennimmt und
 - optional einen Wert zurückgibt.
 - **Parameter und Rückgabewert** werden durch ein **funktionales Interface** festgelegt.
 - **Jedes Lambda basiert auf einem funktionalen Interface.**
- Lambda-Ausdrücke sind ähnlich wie Methoden, sie
 - benötigen **keine Namen** (Implementation ist anonym, Methodename kommt vom Interface),
 - werden **nicht sofort ausgeführt** (sind lazy),
 - sind (möglichst) **zustandslos** (pure),
 - können direkt **innerhalb einer anderen Methode implementiert** werden.

Functional Interfaces

Lambdas basieren in Java auf [Functional Interfaces](#), das sind Interfaces mit **genau einer abstrakten** (genau eine nicht implementierte) Methode.

```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}
```

Anonyme Klasse statt Lambda

Bevor es Lambdas in Java gab, konnten Interfaces mithilfe von anonymen Klassen implementiert werden.

```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}

public static void main(String[] args) {
    // Das Interface als anonyme Klasse implementiert.
    MeaningOfLife meaningOfLife = new MeaningOfLife() {
        @Override
        public int calculate() {
            return 42;
        }
    };

    // Führe die Methode in der anonymen Klasse aus.
    System.out.println(meaningOfLife.calculate());
}
```

Ausgabe:
42

Lambda mit Rückgabewert

Lambdas sind deutlich kürzer als anonyme Klassen. Die Lambda-Implementierung wird durch einem Pfeil -> eingeleitet. Wenn es keine Parameter gibt, muss eine leere Klammer () vor dem -> stehen.

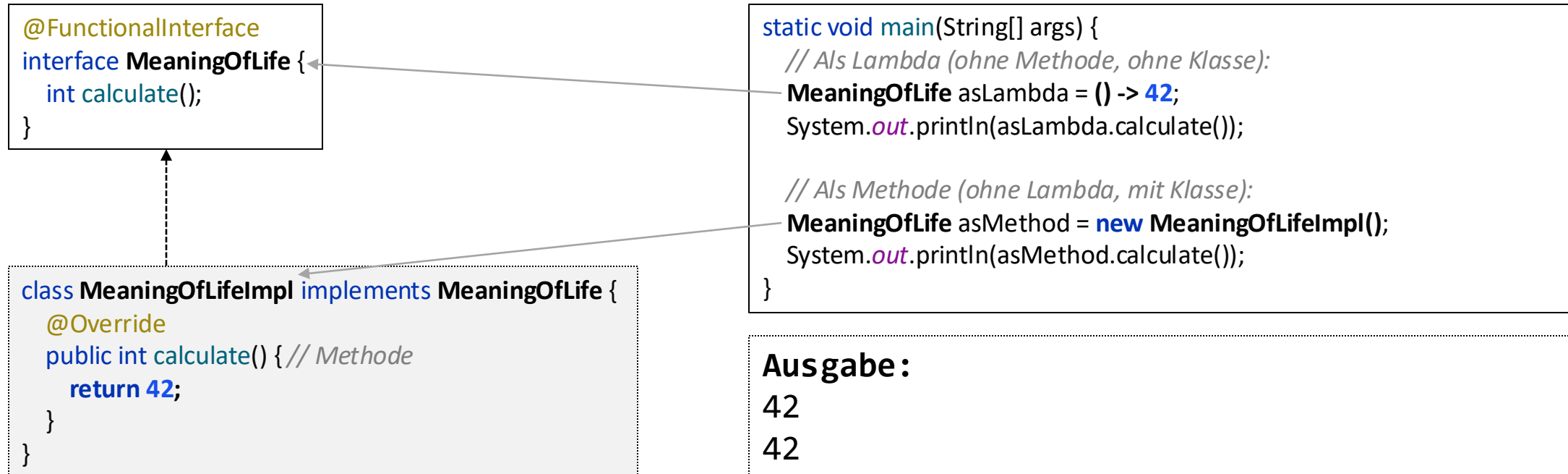
```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}

public static void main(String[] args) {
    // Definiere, was das Lambda tun soll (basierend auf dem MeaningOfLife-Interface).
    MeaningOfLife meaningOfLife = () -> 42;

    // Führe das Lambda aus.
    System.out.println(meaningOfLife.calculate());
}
```

Ausgabe:
42

Implementation: Lambda vs Methode



Als Methode würde man eine Klasse erzeugen, um das gewünschte Interface zu implementieren.

Als Lambda kann man sich den ersten Schritt (eine Klasse erzeugen) sparen.

Lambda mit Parameter

Parameter werden bei Lambdas in den Runden Klammern angegeben. Java kann die Datentypen für die Lambda-Parameter automatisch ermitteln, daher kann man die Datentypen auch weglassen. 🤖

```
@FunctionalInterface
public interface MoneyPrinter {
    void amount(int value);
}

public static void main(String[] args) {
    // Definiere, was das Lambda mit dem Parameter tun soll.
    MoneyPrinter euroPrinter = (int value) -> System.out.print("€" + value);
    MoneyPrinter dollarPrinter = (int value) -> System.out.print("$" + value);

    // Führe das Lambda aus.
    euroPrinter.amount(42);
    dollarPrinter.amount(42);
}
```

Ausgabe: €42 \$42

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 1: *parameter -> expression* oder *{ code block; }*

Beispiel: `persons.forEach(person -> System.out.println(person.name));`

Beispiel: `persons.forEach(person -> { System.out.println(person.name); });`

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 2: *(parameter) -> expression* oder *{ code block; }*

Beispiel: `persons.forEach((person) -> System.out.println(person.name));`

Beispiel: `persons.forEach((person) -> {
 System.out.println(person.name);
});`

Beispiel: `persons.forEach((Person person) -> System.out.println(person.name));`



Der Parametertyp wird in der Regel weggelassen.

Die Schreibweise mit Parametertyp erfordert runde Klammern.

Lambda – Schreibweisen

Klammern sind nötig, wenn mehr als ein Parameter oder Expression verwendet werden.

Ab zwei Parametern sind die runden Klammern () Pflicht.

Variante 3:

(parameter1, parameter2) -> expression oder { code block; }

Beispiel:

```
persons.stream().sorted((p1, p2) -> p1.name.compareTo(p2.name));
```

Beispiel:

```
persons.stream().sorted((p1, p2) -> {  
    return p1.name.compareTo(p2.name);  
});
```

Bei der Kurzschreibweise ohne { } muss man das return weglassen.

Lambda – Schreibweisen

Es gibt eine weitere Schreibweise für Lambdas mit [Methodenreferenz](#).

Variante 4: ***Klassennamen::methodName(parameter)***

Lambda: `numbers.forEach(number -> System.out.println(number));`

Methodenreferenz: `numbers.forEach(System.out::println);`

Lambda: `List.of("a", "b", "c").forEach(letter -> letter.toUpperCase());`

Methodenreferenz: `List.of("a", "b", "c").forEach(String::toUpperCase);`

Lambda: `numbers.forEach(number -> new Person(number));`

Methodenreferenz: `numbers.forEach(Person::new)`

Lambda: `numbers.stream().sorted((n1, n2) -> n1.compareTo(n2));`

Methodenreferenz: `numbers.stream().sorted(Integer::compareTo);`

Lambda – Methodenreferenz Einschränkungen 1/2

Methodenreferenzen sind oft kürzer als die Lambda-Schreibweise, allerdings sind Methodenreferenzen nicht möglich, sobald auf einem Parameter zugegriffen wird.

```
public static void main(String[] args) {  
  
    // 1. Kein Zugriff auf den Parameter.  
    List.of(new Student("Ada")).forEach(student -> System.out.println(student));  
    List.of(new Student("Ada")).forEach(System.out::println); // Äquivalent mit dem obigen Lambda.  
  
    // 2. Zugriff auf dem Parameter. Schreibweise als Methodenreferenz nicht möglich.  
    List.of(new Student("Ada")).forEach(student -> System.out.println(student.name()));  
    // Keine Methodenreferenz-Schreibweise zum obigen Lambda möglich.  
}
```

Lambda – Methodenreferenz Einschränkungen 2/2

Methodenreferenzen sind nur auf implementierte Methoden möglich, aber nicht auf abstrakte Methoden oder Interfacemethoden ohne Implementierung.

@FunctionalInterface

interface MeaningOfLife {

int calculate(); *// Methode ohne Implementierung.*

static int calculateStatic() { *// Methode mit Implementierung.*

return 42;

}

public static void main(String[] args) {

// 1. Methodenreferenz funktioniert nicht mit abstrakten (nicht-implementierten) Methoden.

// MeaningOfLife lambda = MeaningOfLife::calculate; // nicht kompilierbar

// 2. Methodenreferenz funktioniert nur mit implementierten Methoden.

MeaningOfLife lambda = MeaningOfLife::calculateStatic;

System.out.println(lambda.calculate());

}

}

Lambda – Warum?

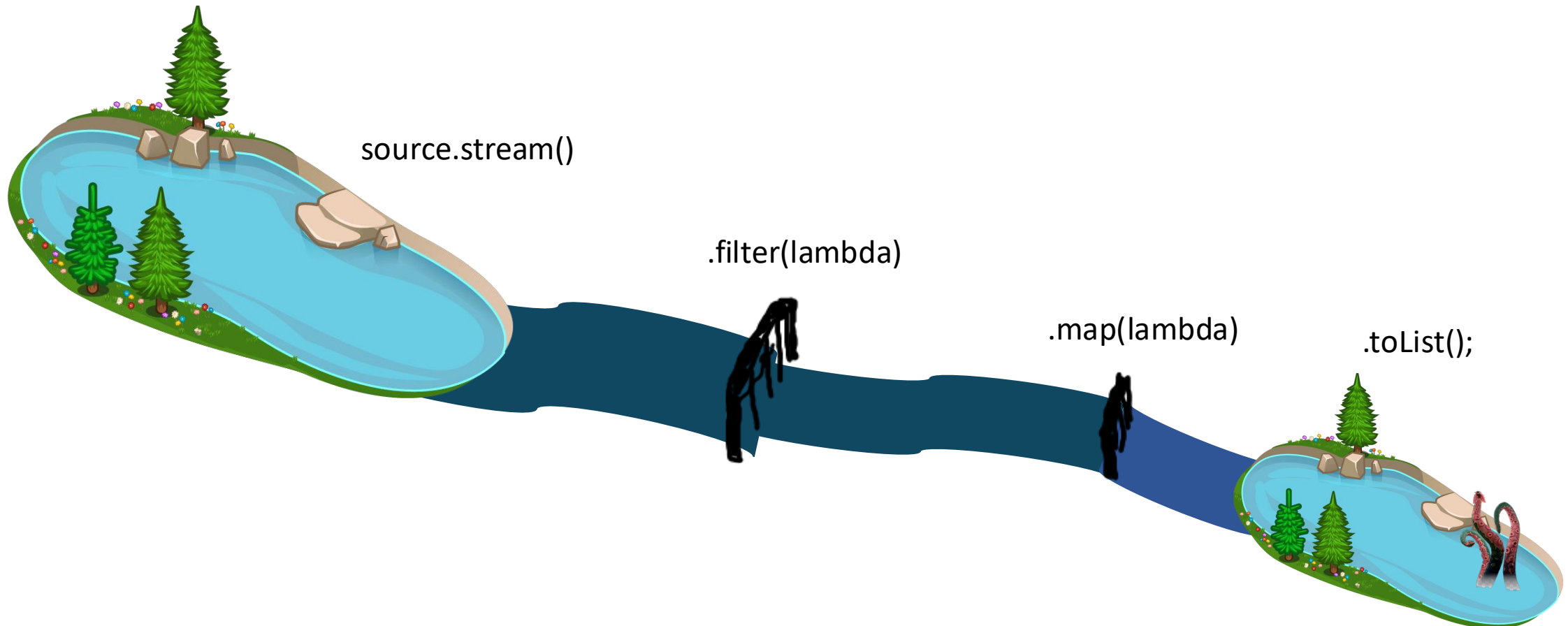
Lambdas lassen sich flexibel „on-the-fly“ erzeugen, um eine Interface-Implementierung **ad-hoc oder temporär zu erzeugen**.

Siehe Streams...

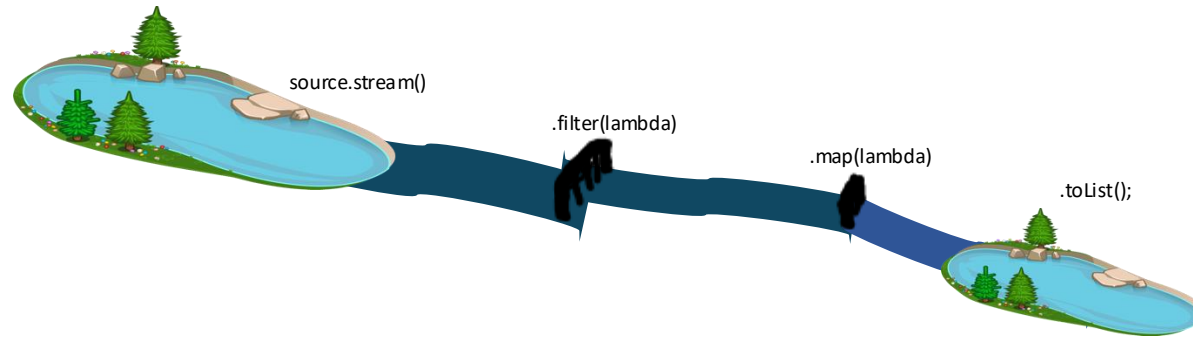
Stream

Stream

Das Interface [Stream<T>](#) aus `java.util.stream` beschreibt einen „Strom“ von Elementen. Dieser „fließt“ von einer Quelle zu einem Ziel. Auf seinem Weg kann der Strom nahezu beliebig manipuliert werden.



Stream



Stream-Erstellung

- `collection.stream()`
- `Stream.of(...)`
- ...

Intermediate Operatoren

- `.filter(lambda)`
- `.map(lambda)`
- `.flatMap(lambda)`
- `.sorted(lambda)`
- ...

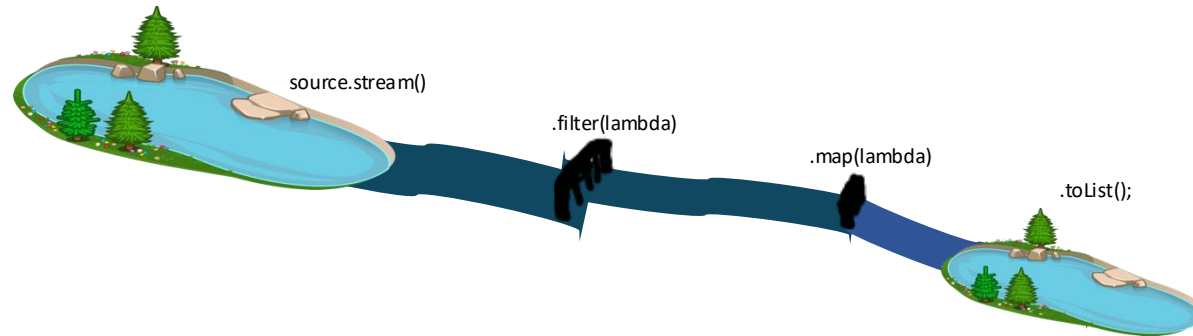
Terminale Operatoren

- `.toList()`
- `.collect(...)`
- `.toList()`
- `.sum()`
- `.reduce()`
- ...



```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```

Stream



Stream-Erstellung

- collection.stream()
- Stream.of(...)
- ...

Intermediate Operatoren

- .filter(lambda)
- .map(lambda)
- .flatMap(lambda)
- .sorted(lambda)
- ...

Terminale Operatoren

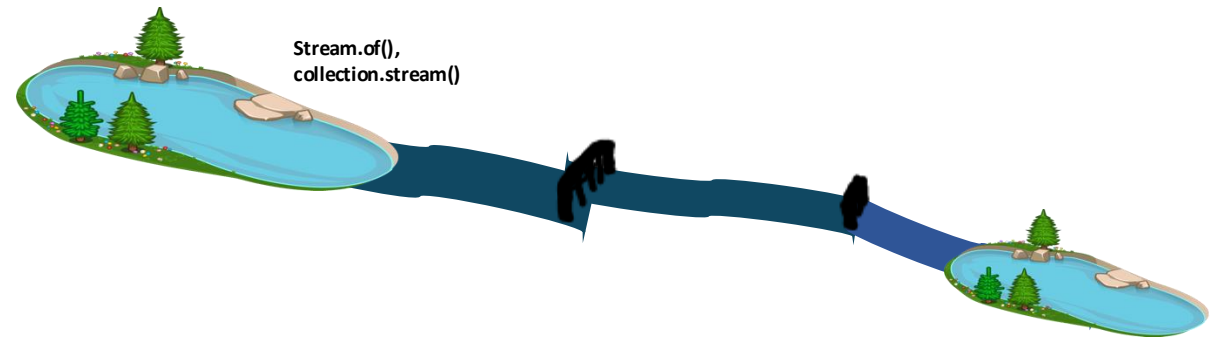
- .toList()
- .collect(...)
- .toList()
- .sum()
- .reduce()
- ...



```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```

🤖 toList() gibt eine nicht veränderbare List zurück (siehe [unmodifiable in Collections](#)).

D. h. die Ergebnisliste kann nicht verändert werden z. B. durch add, remove, set (UnsupportedOperationException)!



Stream erzeugen

```
Stream<Integer> numbers1 = Stream.of(1, 2, 3); // intern wird eine temporäre Liste erzeugt
```

```
Stream<Integer> numbers2 = List.of(1, 2, 3).stream();
```

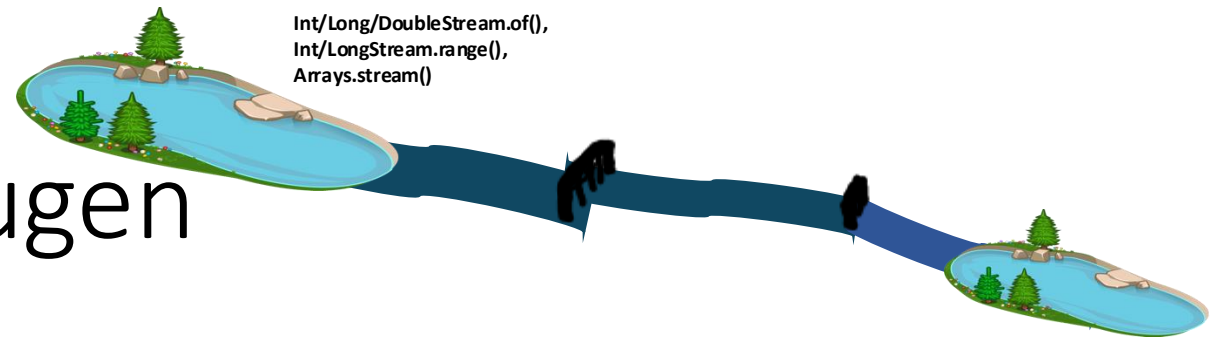
```
Stream<Integer> numbers3 = new ArrayList<>(List.of(1, 2, 3)).stream();
```

```
Stream<String> names1 = Stream.of("Alan", "Adele", "Linus"); // temporäre Liste intern
```

```
Stream<String> names2 = List.of("Alan", "Adele", "Linus").stream();
```

```
Stream<String> names3 = new ArrayList<>(List.of("Alan", "Adele", "Linus")).stream();
```

🤖 Ein Stream selbst **speichert keine Elemente**, sondern greift auf eine vorhandene Datenquelle zu.
Ein Stream verarbeitet ausschließlich **Objekte** (d. h., auch **null** ist möglich).



Primitiven Stream erzeugen

IntStream, LongStream und DoubleStream sind spezielle Streams, die **keine Objekte** sondern **primitive Daten** speichern (kein `null` möglich). Diese Streams können **effizienter und performanter** sein, sobald sehr viele Elemente vorhanden sind (da keine Objekte verwaltet werden müssen, d. h., weniger Overhead).

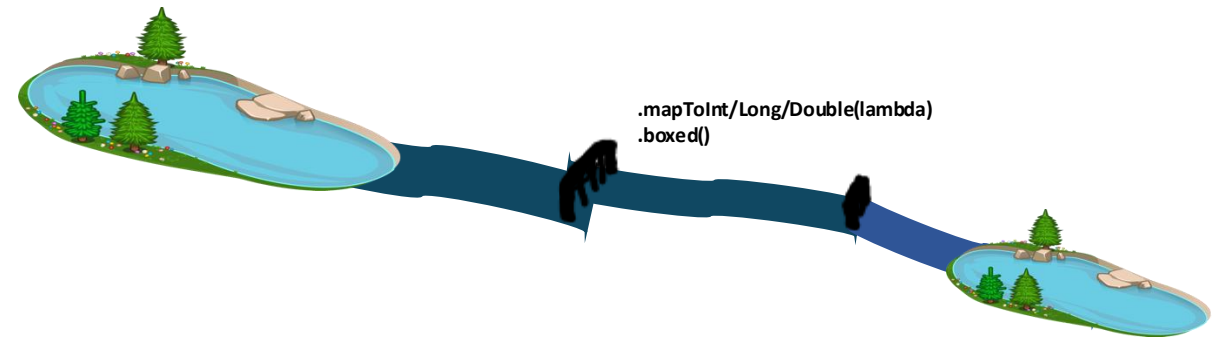
```
IntStream    ints1    = IntStream.of(1, 2, 3);
IntStream    ints2    = IntStream.range(1, 4);    // gleich wie ints1
LongStream   longs1   = LongStream.of(1, 2, 3);
LongStream   longs2   = LongStream.range(1, 4);   // gleich wie longs1
DoubleStream doubles  = DoubleStream.of(1, 2, 3); // kein range möglich
```

Alternativ kann man primitive Streams mit einem vorhandenen Array über `Arrays.stream(array)` erzeugen:

```
IntStream    ints3     = Arrays.stream(new int[] {1, 2, 3});
LongStream   longs3    = Arrays.stream(new long[] {1, 2, 3});
DoubleStream doubles2  = Arrays.stream(new double[] {1, 2, 3});
```

Weiterhin verfügen primitive Streams über **zusätzliche oder spezialisierte Methoden** wie z. B. `min()`, `max()`, `average()`, `sum()`, `mapToInt/Long/Double/Obj()`.

Streams umwandeln



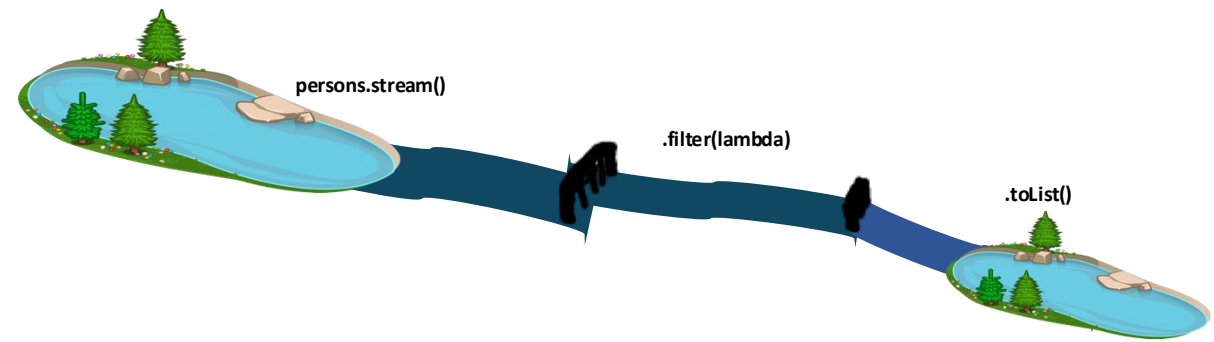
Um einen primitiven Stream **in einem Objekt-Stream umzuwandeln**, kann **mapToObj()** oder **boxed()** verwendet werden (jedes primitive Element wird in ein entsprechendes Objekt gepackt).

```
Stream<Integer> intObjects    = IntStream.of(1, 2, 3).boxed();
Stream<Long>     longObjects   = LongStream.of(1, 2, 3).boxed();
Stream<Double>   doubleObjects = DoubleStream.of(1, 2, 3).boxed();
```

Ein Objekt-Stream kann mithilfe von `mapToInt/Long/Double(lambda)` in einen primitiven Stream umgewandelt werden.

```
IntStream      ints          = Stream.of(1l, 2l, 3l).mapToInt(l -> l.intValue());
LongStream     longs         = Stream.of(1d, 2d, 3d).mapToLong(d -> d.longValue());
DoubleStream   doubles       = Stream.of(1, 2, 3).mapToDouble(i -> i);
```


Stream – Beispiel 1



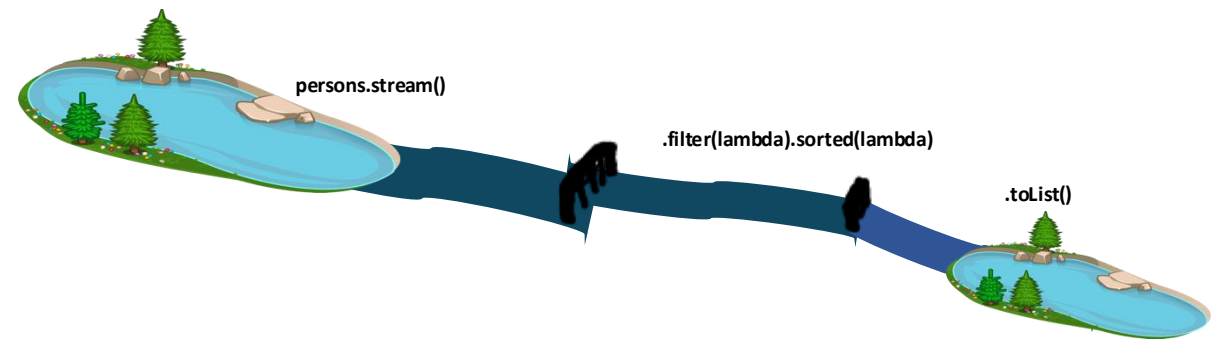
```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        List<Person> minors = persons.stream()  
            .filter(person -> person.age() < 18)  
            .toList();  
        System.out.println(minors);  
    }  
}
```

Ausgabe:

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15]]
```

person.age() ist ein Getter aus dem record Person und gibt ein int zurück.

Stream – Beispiel 2



```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        List<Person> minors = persons.stream()  
            .filter(person -> person.age() < 18)  
            .sorted((person1, person2) -> person1.name().compareTo(person2.name()))  
            .toList();  
        System.out.println(minors);  
    }  
}
```

Ausgabe:

```
[Person[name=Alice, age=15],  
Person[name=Bob, age=10]]
```

person1.name() ist ein Getter aus dem record Person und gibt einen String zurück.

compareTo ist eine Methode aus der Klasse String (die das Comparable-Interface implementiert) und gibt ein int zurück, der die Reihenfolge bestimmt.



Stream – Essentielles Wissen

- Ein [Stream](#) selbst **speichert keine Elemente**. Er bezieht sie aus einer **Quelle**. Typische Quellen sind [Collections](#) (z. B. ArrayList, HashSet, Arrays), aber auch andere Quellen sind möglich.
- Nach der Erzeugung eines Streams, können **beliebig** viele **intermediate Operationen** zum Stream hinzugefügt werden.
- Ein Stream ist im Prinzip eine **Sammlung von auszuführenden Funktionen**.
- Streams sind **lazy**. Die enthaltenen **Funktionen** werden erst **durch Aufruf einer terminalen Operation nacheinander ausgeführt**.
- Ein Stream wird durch eine terminale Operation geschlossen. Der Stream kann anschließend **nicht wiederverwendet** werden!

filter(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// Oldschool -----

```
List<Person> minors = new ArrayList<Person>();  
for (Person person : persons)  
    if (person.age() < 18)  
        minors.add(person);  
System.out.println(minors);
```

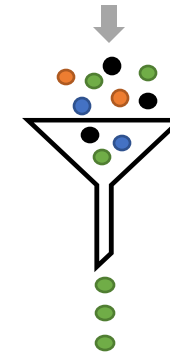
// Stream -----

```
minors = persons.stream()  
    .filter(person -> person.age() < 18)  
    .toList();  
System.out.println(minors);  
}  
}
```

Merkhilfe:

to filter: etwas ausfiltern

„Gib mir nur die Elemente zurück,
die ich behalten will.“



Ausgabe:

```
[Person[name=Bob, age=10], Person[name=Alice, age=15]]  
[Person[name=Bob, age=10], Person[name=Alice, age=15]]
```

map(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
    }  
}
```

// Oldschool -----

```
List<String> modifiedNames = new ArrayList<String>();  
for (Person person : persons) {  
    String modifiedName = person.name().toUpperCase();  
    modifiedNames.add(modifiedName);  
}  
System.out.println(modifiedNames);
```

// Stream -----

```
modifiedNames = persons.stream()  
    .map(person -> person.name().toUpperCase()) // Es wird im Lambda von map ein String zurückgegeben, deswegen  
    .toList(); // enthält das Ergebnis der map-Methode eine Liste von Strings  
System.out.println(modifiedNames); // und keine Liste von Personen (List<String> statt List<Person>)!  
}
```

Merkhilfe:

to map: etwas abbilden

„Ich mappe mir die Welt, wie sie mir gefällt!“
Aus Objekt Person("Bob", 10) wird String "BOB"

Ausgabe:

```
[BOB, ALICE, JANE]  
[BOB, ALICE, JANE]
```

flatMap(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<List<Person>> teams = List.of(  
            List.of(new Person("Bob", 10), new Person("Alice", 15)), // Sublist 1  
            List.of(new Person("Jane", 20), new Person("Anna", 17)) // Sublist 2  
        );
```

// Oldschool -----

```
List<Person> persons = new ArrayList<>();  
for (List<Person> team : teams) {  
    for (Person person : team) { // Alternativ: persons.addAll(team);  
        persons.add(person);  
    }  
}  
System.out.println(persons);
```

// Stream -----

```
persons = teams.stream().flatMap(person -> person.stream()).toList();  
// persons = teams.stream().flatMap(List::stream).toList();  
System.out.println(persons);  
}  
}
```



Hinweis: Stream::flatMap erwartet, dass das Lambda einen Stream zurückgibt.

Merkhilfe:

to flatMap: "flachklopfen"

Mache aus einer "Liste von Listen" eine Liste.

Ausgabe:

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15],  
Person[name=Jane, age=20],  
Person[name=Anna, age=17]]
```

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15],  
Person[name=Jane, age=20],  
Person[name=Anna, age=17]]
```

mapToInt/Long/Double(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// Oldschool -----

```
int sumAge = 0;  
for (Person person : persons)  
    sumAge = sumAge + person.age();  
System.out.println(sumAge);
```

// Stream -----

```
sumAge = persons.stream()  
    .mapToInt(person -> person.age()) // mapToInt ist eine spezielle Variante von map.  
    // .mapToInt(Person::age())        // Alternative Schreibweise mittels Methodenreferenz.  
    .sum();                            // mapToInt liefert einen IntStream zurück, der u. a. die sum()-Methode enthält.  
System.out.println(sumAge);  
}  
}
```

Merkhilfe:

mapToInt liefert ein IntStream zurück, der zusätzliche Methoden enthält, z. B. min, max, sum und average.

Ausgabe:

45

45

IntStream.range(lambda) und indexOf

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

```
// Old school -----  
int indexOfAlice = -1;  
for (int i = 0; i < persons.size(); i++) {  
    Person person = persons.get(i);  
    if (person.name().equals("Alice")) {  
        indexOfAlice = i;  
        break;  
    }  
}  
System.out.println(indexOfAlice);
```

```
// Statt old school einfacher mit List::indexOf-Methode:  
indexOfAlice = persons.indexOf(new Person("Alice", 23));
```

```
// Stream -----  
indexOfAlice = IntStream.range(0, persons.size())  
    .filter(i -> persons.get(i).name.equals("Alice"))  
    .findFirst().orElse(-1);  
System.out.println(indexOfAlice);  
}  
}
```

Dies ist zurzeit die "eleganteste" Lösung, um mit Streams den Index von einem gesuchten Objekt zu erhalten.

Ausgabe:

1
1
1

sorted(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

```
// Oldschool -----  
List<Person> sortedPersons = new LinkedList<Person>(persons);  
sortedPersons.sort((p1, p2) -> Integer.compare(p1.age(), p2.age()));  
// Collections.sort(sortedPersons, (p1, p2) -> Integer.compare(p1.age(), p2.age()));  
System.out.println(sortedPersons);
```

```
// Stream -----  
sortedPersons = persons.stream()  
    .sorted((p1, p2) -> Integer.compare(p1.age(), p2.age()))  
    // .sorted(Comparator.comparingInt(Person::age)) // Alternative Schreibweise.  
    .toList();  
System.out.println(sortedPersons);  
}  
}
```

Merke:

sorted erhält zwei Parameter, die miteinander verglichen werden und benötigt ein int als Rückgabe für die Reihenfolge.

Ausgabe:

```
[Person[name=Jane, age=1],  
Person[name=Bob, age=2],  
Person[name=Alice, age=3]]
```

```
[Person[name=Jane, age=1],  
Person[name=Bob, age=2],  
Person[name=Alice, age=3]]
```

skip/limit(lambda)

intermediate operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 2), // skipped  
            new Person("Alice", 3),  
            new Person("Jane", 1),  
            new Person("Ada", 4) // limited  
        );  
    }  
}
```

// Oldschool -----

```
List<Person> result = new ArrayList<>();  
int skip = 1;  
int limit = 2;  
for (Person person : persons) {  
    if (skip-- > 0)  
        continue;  
    if (limit-- == 0)  
        break;  
    result.add(person);  
}  
System.out.println(result);
```

// Stream -----

```
result = persons.stream()  
    .skip(1) // überspringe die ersten x Elemente.  
    .limit(2) // gebe maximal x Elemente zurück.  
    .toList();  
System.out.println(result);  
}
```

Merke:

*Überspringe die ersten x Elemente (skip) oder
gebe maximal x Elemente (limit) zurück.*

Ausgabe:

```
[Person[name=Alice, age=3], Person[name=Jane, age=1]]  
[Person[name=Alice, age=3], Person[name=Jane, age=1]]
```

toList(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        // Stream -----  
        List<Person> newPersons = persons.stream().toList(); // nicht modifizierbar  
        newPersons.add(new Person("Ada", 10)); // wirft Exception zur Laufzeit  
    }  
}
```

Merkhilfe:

Gibt eine nicht-modifizierbare Liste zurück. D. h., add-, remove-, set-Methoden werfen zur Laufzeit eine UnsupportedOperationException.

Ausgabe:

UnsupportedOperationException

collect(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        // Stream -----  
        List<Person> newPersons = persons.stream().collect(Collectors.toList());  
        newPersons.add(new Person("Ada", 2)); // eine modifizierbare Liste  
  
        List<Person> newPersonsSet = persons.stream().collect(Collectors.toSet());  
        newPersonsSet.add(new Person("Bob", 10)); // ein modifizierbares Set  
    }  
}
```

Merkhilfe:

Ermöglicht es, die Rückgabe eine Streams beliebig anzupassen.

Die Klasse [Collectors](#) enthält hierzu viele Hilfsmethoden.

forEach(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// Oldschool -----

```
for (Person person : persons)  
    System.out.println(person.name());
```

// Stream -----

```
persons.forEach(person ->  
    System.out.println(person.name()));  
}  
}
```

Achtung: Verwechsle forEach nicht mit map!

forEach gibt keine Objekte zurück (void) und ist eine terminale Methode, während map eine Liste mit modifizierten Objekten zurückgibt und eine intermediate Operation ist.

Merkhilfe:

Alternative zur for-Schleife.

forEach ist eine terminale Methode.

Verwechsel forEach nicht mit map:

map ist intermediate und gibt

modifizierte Werte zurück. forEach

hat keinen Rückgabewert (void)!

Ausgabe:

Bob

Alice

Jane

Bob

Alice

Jane

reduce(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// Oldschool -----

```
int sumAge = 0;  
for (Person person : persons)  
    sumAge = sumAge + person.age;  
System.out.println(sumAge);
```

// Stream -----

```
sumAge = persons.stream()  
    .map(person -> person.age) // map wandelt die Liste von Personen in eine Liste von Zahlen (age) um.  
    .reduce(0, (subtotal, age) -> subtotal + age); // subtotal wird mit 0 initialisiert. age ist jede Zahl aus der map-Methode.  
System.out.println(sumAge);  
}  
}
```

Merkhilfe:

*to reduce: etwas verkleinern
„Reduziere die Liste zu einem
einigen Wert.“*

[1 2 3 4 5] → 15

Ausgabe:

45

45

any/all/noneMatch(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        // Oldschool -----  
        boolean hasAnyPersonIceInName = false;  
        for (Person person : persons) {  
            if (person.name().contains("ice")) {  
                hasAnyPersonIceInName = true;  
                break;  
            }  
        }  
        System.out.println(hasAnyPersonIceInName);  
  
        // Stream -----  
        hasAnyPersonIceInName = persons.stream()  
            .anyMatch(person -> person.name().contains("ice"));  
        System.out.println(hasAnyPersonIceInName);  
    }  
}
```

Merkhilfe:

to match: übereinstimmen

„Ist das vorhanden?“

→ true oder false

Ausgabe:

true

true

min/max(lambda)

terminal operation

```
record Person(String name, int age) {  
    public static void main(String[] args) {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

// Oldschool -----

```
Person result = null;  
for (Person person : persons) {  
    if (result == null || person.age() < result.age()) {  
        result = person;  
    }  
}  
System.out.println(result);
```

// Stream -----

```
Optional<Person> optionalResult = persons.stream()  
    .min((p1, p2) -> Integer.compare(p1.age(), p2.age()));  
// .min(Comparator.comparingInt(Person::age)); // Alternative Schreibweise.  
System.out.println(optionalResult);  
}
```

Merke:

min/max erhält zwei Parameter, die miteinander verglichen werden und benötigt ein int als Rückgabe für die Reihenfolge:

- **negativ Zahl:** Element 1 kleiner als Element 2
- **positive Zahl:** Element 1 größer als Element 2
- **0:** Element 1 ist gleich wie Element 2
(ursprüngliche Reihenfolge bleibt erhalten)

Ausgabe:

```
Person[name=Jane, age=1]  
Optional[Person[name=Jane, age=1]]
```


java.util.Optional<T>

Manche Stream-Methoden geben ein Optional<T> zurück.

Beispiel:

```
System.out.println(IntStream.of(1, 2, 3).average());
```

Ausgabe: OptionalDouble[2.0]

Gibt ein Optional (hier sogar ein spezielles OptionalDouble) zurück, da bei average() theoretisch eine "Division by Zero" möglich ist, falls der Stream leer ist:

```
System.out.println(IntStream.of().average());
```

Ausgabe: OptionalDouble.empty

Hinweis:

Stream-Methoden, die ein Optional<T> zurückgeben, sind zum Beispiel: findFirst, findAny, min, max, reduce, und Int/Long/DoubleStream::average

java.util.Optional<T>

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
System.out.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
    System.out.println(greeting.toLowerCase());
```



java.util.Optional<T>.get()

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
System.out.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
—System.out.println(greeting.toLowerCase());
```



// Mit Optional<T>:

```
import java.util.Optional;  
Optional<String> greeting = Optional.ofNullable(aVariableThatMightBeNull);
```

// Beispiel bad practice:

```
if (greeting.isPresent()) // kein Gewinn zu if(greeting != null)  
    System.out.println(greeting.get());
```



java.util.Optional<T>.orElse(defaultValue)

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
System.out.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
— System.out.println(greeting.toLowerCase());
```



// Mit Optional<T>:

```
import java.util.Optional;  
Optional<String> greeting = Optional.ofNullable(aVariableThatMightBeNull);
```

// Beispiel bad practice:

```
if (greeting.isPresent()) // kein Gewinn zu if(greeting != null)  
— System.out.println(greeting.get());
```



// Beispiel good practice:

```
greeting.orElse("Hi"); // return Wert oder Default-Wert wenn null  
greeting.orElseThrow(() -> IllegalArgumentException::new) // return Wert oder Exception wenn null
```



Stream vs for-Loop (oldschool)

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3, 5, 4, 7, 8, 9);
```

```
    // Variante 1: for-Loop (oldschool)
```

```
    int result = 0;  
    for (int e : numbers) {  
        if (e > 3 && e % 2 == 0) {  
            result = e * 2;  
            break;  
        }  
    }  
    System.out.println(result);
```

```
    // Variante 2: Stream
```

```
    result = numbers.stream()  
        .filter(e -> e > 3)  
        .filter(e -> e % 2 == 0) // FYI: man könnte beide Filtermethoden auch mit && zusammenfassen.  
        .map(e -> e * 2)  
        .findFirst()  
        .orElse(0); // Keine Stream-Methode sondern aus der Klasse Optional<T>  
    System.out.println(result);  
}
```

Ausgabe:

8
8

Im Vergleich: Streams sind ausdrucksstärker und beschreiben mehr, was der Code macht.
Und das bei gleicher Performance aufgrund von Lazy Evaluation! 🤓

Vorteile Streams

- Kompakterer Code
- Leichtere Lesbarkeit
- Keine Nebenwirkungen
- Einfach kombinierbare Operationen
- Gleiche oder bessere Performance
- Streams zwingen uns, **funktional zu denken**:
→ „*Was möchte ich tun?*“ statt „*Wie gehe ich Schritt für Schritt vor?*“

Anwendungsszenarien

- Filtern, Sortieren, Gruppieren, Aggregieren
- Datenverarbeitung in Pipelines
- Lesbarer und wartbarer Code

Intermediate Operations

Operation	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>.filter()</code>	Filtert Elemente nach einer Bedingung	<code>Stream<Integer></code>	<code>Stream<Integer></code> mit gefilterten Werten	<code>Stream.of(1,2,3,4).filter(n -> n > 2) → [3,4]</code>
<code>.map()</code>	Wandelt jedes Element in ein anderes Objekt um	<code>Stream<Person></code>	<code>Stream<String></code> (z. B. Namen)	<code>persons.stream().map(p -> p.name()) → ["Bob", "Alice", "Jane"]</code>
<code>.flatMap()</code>	Vereinfacht verschachtelte Strukturen („flachklopfen“)	<code>Stream<List<Person>></code>	<code>Stream<Person></code>	<code>teams.stream().flatMap(List::stream)</code>
<code>.sorted()</code>	Sortiert Elemente nach Comparator	<code>Stream<Integer></code>	<code>Stream<Integer></code> sortiert	<code>Stream.of(3,1,2).sorted() → [1,2,3]</code>
<code>.distinct()</code>	Entfernt doppelte Elemente	<code>Stream<Integer></code>	<code>Stream<Integer></code> ohne Duplikate	<code>Stream.of(1,1,2).distinct() → [1,2]</code>
<code>.skip(n)</code>	Überspringt die ersten n Elemente	<code>Stream<String></code>	<code>Stream<String></code> mit Rest	<code>Stream.of("A", "B", "C").skip(1) → ["B", "C"]</code>
<code>.limit(n)</code>	Begrenzt auf n Elemente	<code>Stream<String></code>	<code>Stream<String></code> (erste n)	<code>Stream.of("A", "B", "C").limit(2) → ["A", "B"]</code>
<code>.peek()</code>	Führt Aktion mit jedem Element aus (z. B. Logging), verändert nichts	<code>Stream<T></code>	<code>Stream<T></code> (unverändert)	<code>stream.peek(System.out::println)</code>
<code>.boxed()</code>	Wandelt primitive Werte in Wrapper-Objekte um	<code>IntStream</code>	<code>Stream<Integer></code>	<code>IntStream.of(1,2).boxed() → [1,2]</code>

Terminal Operations

Operation	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>.toList()</code>	Gibt alle Elemente als unveränderbare Liste zurück	<code>Stream<T></code>	<code>List<T></code> (unmodifiable)	<code>Stream.of(1,2).toList() → [1,2]</code>
<code>.collect()</code>	Wandelt Stream in beliebige Sammlung um	<code>Stream<T></code>	<code>Collection<T></code> (z. B. Set, List, Map)	<code>stream.collect(Collectors.toSet())</code>
<code>.reduce()</code>	Kombiniert alle Werte zu einem Ergebnis	<code>Stream<Integer></code>	<code>Integer</code>	<code>Stream.of(1,2,3).reduce(0, Integer::sum) → 6</code>
<code>.forEach()</code>	Führt Aktion für jedes Element aus (kein Rückgabewert)	<code>Stream<T></code>	<code>void</code>	<code>.forEach(System.out::println)</code>
<code>.count()</code>	Zählt Elemente im Stream	<code>Stream<T></code>	<code>long</code>	<code>Stream.of("A", "B").count() → 2</code>
<code>.anyMatch()</code>	Prüft, ob mind. ein Element Bedingung erfüllt	<code>Stream<T></code>	<code>boolean</code>	<code>.anyMatch(x -> x>5) → true</code>
<code>.allMatch()</code>	Prüft, ob alle Bedingung erfüllen	<code>Stream<T></code>	<code>boolean</code>	<code>.allMatch(x -> x>0) → true</code>
<code>.noneMatch()</code>	Prüft, ob kein Element Bedingung erfüllt	<code>Stream<T></code>	<code>boolean</code>	<code>.noneMatch(x -> x<0) → true</code>
<code>.findFirst()</code>	Gibt erstes Element (optional) zurück	<code>Stream<T></code>	<code>Optional<T></code>	<code>.findFirst() → Optional[1]</code>
<code>.min() / .max()</code>	Liefert kleinstes/größtes Element	<code>Stream<T></code>	<code>Optional<T></code>	<code>.min(Comparator.naturalOrder())</code>

Primitive Streams und Optional

Typ / Methode	Beschreibung	Eingabe	Ausgabe	Beispiel
<code>IntStream,</code> <code>LongStream,</code> <code>DoubleStream</code>	Spezielle Streams für primitive Werte	primitive Werte (int, long, double)	Stream entsprechender Typen	<code>IntStream.range(1, 4) → 1, 2, 3</code>
<code>.sum()</code>	Addiert alle Werte	<code>IntStream</code>	<code>int</code>	<code>IntStream.of(1, 2, 3).sum() → 6</code>
<code>.average()</code>	Berechnet Durchschnitt	<code>IntStream</code>	<code>OptionalDouble</code>	<code>IntStream.of(2, 4, 6).average() → OptionalDouble[4.0]</code>
<code>.boxed()</code>	Verpackt primitive Werte als Objekte	<code>IntStream</code>	<code>Stream<Integer></code>	<code>IntStream.of(1, 2, 3).boxed() → [1, 2, 3]</code>
<code>Optional<T></code>	Verpackt evtl. leere Werte	evtl. null	<code>Optional<T></code>	<code>Optional.ofNullable(value)</code>
<code>.orElse()</code>	Gibt Wert oder Default-Wert	<code>Optional<T></code>	<code>T</code>	<code>optional.orElse("default")</code>
<code>.orElseThrow()</code>	Gibt Wert oder Exception	<code>Optional<T></code>	<code>T</code> oder Exception	<code>optional.orElseThrow()</code>